CHAPTER 5

A Closer Look at Instruction Set Architectures

The instruction *consists of opcodes and operands.*

The opcodes specify the operations to be executed.

The operands specify register or memory locations of data.

Instruction Format

The instruction sets are differentiated by the following features:

1- Operand storage in the CPU (data can be stored in stack structure or in registers)

2 – Number of operands per instruction.

3 – Operand location (register to register, register to memory ……).

4 – Operations.

5 - Type and size (operands can be addresses, numbers , or characters).

Internal Storage in the CPU

Stacks vs Registers

The means of storing data by the CPU

1 – A stack architecture.

2 – An accumulator architecture.

3 – A general purpose register (GPR) architecture.

Internal Storage in the CPU

Stacks vs Registers

Stack architectures use a stack to execute the instruction, and operands are found on the top of the stack (good solution if the memory access is fast).

Accumulator architecture (such as in MARIE), with one operand implicitly in the accumulator, minimize the internal complexity of the machine.

Internal Storage in the CPU

Stacks vs Registers

General Purpose Register architectures use sets of general purpose registers, this is the most acceptable model for machines architecture today.

Stack's examples

$$(X + Y) \times (W - Z) + 2$$

nes:

$$XY + WZ - \times 2+$$

infix:	5 + 5	
postfix:	5 5 +	= 10
pretfix:	+ 5 5	

infix:	4 + 7 * 2	
postfix:	4 7 2 * +	= 18
	14	

infix: A / B + 4

postfix: (A B /) (4) +

infix: (A * B) + (C / D)

postfix: (A B *) (C D /) +

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

push

Stack

infix: 5 / 5 + 2 - 1 * 9

5/5

postfix: 5 5 / 2 + 1 9 * -

pop

Stack

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

push

Stack

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

push

Stack

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

pop 1*9

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

push

Stack

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

pop 3-9

Stack

infix: 5 / 5 + 2 - 1 * 9

postfix: 5 5 / 2 + 1 9 * -

suffix or reverse Polish notation

prefix: - + / 5 5 2 * 1 9

Polish notation

push

Stack

Number of operands *and* instruction length.

Instructions can be formatted in two ways:

1 - Fixed length: wastes space, but is fast and results in better performance.

2 – Variable length: More complex but saves storage space.

MARIE uses a fixed length instruction with a 4-bit OPcode and a 12-bit operand.

Number of operands *and* instruction length.

Some common instructions format

- *OPCODE only* (zero addresses)
- *OPCODE + 1 Address* (usually a memory address)
- *OPCODE + 2 Addresses* (usually registers, or one register and one memory address)
- *OPCODE + 3 Addresses* (usually registers, or combinations of registers and memory)

MARIE is limited on the maximum number of operands allowed per instruction to one operand only.

Some instructions have no operands (as Halt instruction).

Number of operands *and* instruction length.

All architectures have a limit number of operands / instruction.

In MARIE, the maximum is one operand.

Some instructions have no operands (Halt).

Number of operands *and* instruction length.

In Stack

One operand: `Push X` places the data value found at memory location *X* onto the stack; `Pop X` removes the top element in the stack and stores it at location *X*.

For operations requiring two operands, the top two elements of the stack are used. For example, if we execute an `Add` instruction, the CPU adds the top two elements of the stack, popping them both and then pushing the sum onto the top of the stack.

Last Input-First Output (LIFO) ordering is used.

EXAMPLE 5.1 Suppose we wish to evaluate the following expression:

$$Z = (X \times Y) + (W \times U)$$

Three address instructions

```
Mult    R1, X, Y
Mult    R2, W, U
Add     Z, R2, R1
```

1 register + 2 memory addresses

2 registers + 1 memory addresses

Typically when three operands are allowed, at least one operand must be a register.

First operand is normally the ***destination****.*

Two address instructions

Using two instructions , the code will be:

```
Load    R1, X
Mult    R1, Y
Load    R2, W
Mult    R2, U
Add     R1, R2
Store   Z, R1
```

One address specify the register and the other operand could be either a register or a memory address.

One address instruction

```
Load    X       In accumulator
Mult    Y
Store   Temp
Load    W       In accumulator
Mult    U
Add     Temp
Store   Z       In memory
```

Using one address instructions (as in MARIE), we must assume a register (normally the accumulator) is implied as the destination for the result of the instruction.

Stack

The program with a stack-based machine with zero address instructions.(no operands)

```
Push    X
Push    Y
Mult
Push    W
Push    U
Mult
Add
Store   Z
```

No operands
also ***Subt, Divide***

Push places the operand on the top of the stack. Pop removes the stack top and places it in the operand.

Instruction Types

The instructions are grouped into the following categories:

- Data movement
- Arithmetic
- Boolean
- Bit manipulation (shift and rotate)
- I/O
- Transfer of control
- Special purpose